CS201 Midterm 3

22 May 2024

Olcay Taner YILDIZ

***Abstract*— Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures including arrays, (iv) you can use any function given in the data structure, (v) you can not modify the data structure contents.**

I. Question (Disjoint Set) (17 pts)

Write the method in **DisjointSet** class

```
int* numberOfDescendants(int index)
```

which returns the descendants (children, grandchildren, grandgrandchildren, etc.) of set with index $index$. Your method should run in $\mathcal{O}(N)$ time. The size of the returning array should be as much as needed.

II. Question (Disjoint Set) (12 pts)

Rewrite constructor in **DisjointSet** class

```
DisjointSet(int count)
```

such that all numbers with a common factor other than 1 will be in the same set. The numbers are from 0 to count - 1.

III. Question (Heap) (17 pts)

Write the method in **MinHeap** class

```
int howManyChildrenCanBeSwapped()
```

that returns the number of nodes in the heap whose children can be swapped without hurting the heap property. Your method should run in $\mathcal{O}(N)$ time.

IV. Question (Heap) (21 pts)

Write the method in **MaxDHeap** class

```
int third()
```

that returns the third maximum number in the heap. Your method should run in $\mathcal{O}(d^2)$ time.

V. Question (Graph) (21 pts)

Write the method in linked list implementation

```
int* twoHops(int index)
```

which returns the indexes of the nodes which are reachable by two hops, that is, it will consist of all indexes j, where one goes from index node to node i, then from node i to node j. The size of the returning array should be as much as needed. If there are two or more ways to go to a node j, then j must appear that many times in the list (no need to sort or check for duplicates).

VI. Question (Graph) (12 pts)

Write the method in array implementation

```
bool isSubGraph(const Graph& g)
```

which returns true if g is a subgraph of the current graph, false otherwise. A graph G_1 is a subgraph of graph G_2 if every edge of graph G_1 is also an edge in graph G_2.